ASSIGNMENT – UNIT 5

NoSQL Database | Riak Key-Value Store and Neo4j Graph Database

1. Explain the architecture and working of Riak as a Key-Value NoSQL database.

Riak is a distributed key-value database designed for high availability and fault tolerance. Data is stored as an object identified by a unique key inside a bucket (a logical namespace). An object can contain a value, metadata, and content type. For example, bucket “students” may contain key “S101” with a value holding a student’s details.

Riak distributes keys across a cluster using consistent hashing. The key’s hash determines the primary partition, while replicas are stored on other nodes. When a client writes or reads data, Riak routes the request to the responsible nodes. Replication helps keep data available if a node fails. Configurable read and write quorums determine how many replicas must respond before an operation succeeds.

Unlike a relational database, Riak does not require a fixed table schema or joins. It is designed for distributed operation, horizontal scaling, and availability. Its simple key-based access is fast, but complex relationships and multi-record transactions are not its main strengths.

2. Explain the Key-Value Data Model in detail.

A key-value store keeps each item as a pair: a unique key and its associated value. The key is used to locate the value; the value may be text, JSON, binary data, or an application-defined structure. For example, “session:8392” can map to a JSON value containing a user ID, login time, device, and expiry.

Core features include simple access by key, flexible value formats, low-latency reads and writes, and easy distribution across many servers. The usual access pattern is GET(key), PUT(key, value), and DELETE(key). Some systems also support expiration, atomic updates, or limited indexing.

Limitations include limited ad hoc queries, no automatic joins, and difficulty enforcing relationships or updating many keys atomically. Key-value databases suit fast lookups because the system can route directly from a key to its stored value without scanning or joining multiple tables. They are common for sessions, caching, carts, preferences, and configuration data.

3. Analyze the consistency model of Riak.

Riak is designed to remain available in a distributed environment and lets applications configure how many replicas must acknowledge reads and writes. Common parameters include N (replica count), R (read acknowledgements), and W (write acknowledgements). Larger R and W values can improve the chance that a read sees a recent write, but may increase latency or reduce availability during failures.

In an asynchronous distributed system, replicas can temporarily disagree. Riak can resolve concurrent updates through conflict-resolution mechanisms, including application-supplied resolution or data types designed for concurrent updates. The CAP theorem says that when a network partition occurs, a distributed system must make a trade-off between consistency and availability; partition tolerance is necessary when partitions are possible.

Example: an online store has three replicas of a cart. During a network split, one side may accept an update while another side also accepts a change. When connectivity returns, the system must reconcile the versions. A stricter quorum may reject some operations during the split, while a more available configuration may accept writes and resolve conflicts later.

4. Discuss transactions and multi-operation transactions in Riak.

Traditional relational transactions commonly provide ACID guarantees across multiple rows and tables. Distributed key-value databases such as Riak are optimized for independent operations across a cluster, so a transaction spanning many keys can require coordination among different nodes. This adds network delays and makes availability harder during node failures or partitions.

Riak generally emphasizes highly available key-level operations rather than general relational-style multi-key transactions. Applications can maintain correctness by designing operations to be idempotent, using conditional writes or version checks where supported, storing tightly coupled information in one value, and using Riak’s data types for supported atomic updates. For workflows across multiple keys, an application can use a saga: perform a sequence of steps and define compensating actions if a later step fails.

For example, when placing an order, the application may create an order record, update a cart, and reserve inventory. It should track each step’s status, retry safely, and compensate or reconcile failed steps instead of assuming all keys update atomically.

5. Explain the query features of Riak.

Querying by key means the client already knows the key and retrieves the corresponding value directly. Querying by data means searching for records using fields inside values, such as finding all users whose city is “Roorkee.” This is harder because a basic key-value store does not automatically index every field.

Depending on the Riak edition, version, and configuration, query options have included secondary indexes and full-text search integrations. Secondary indexes can support lookups on selected indexed fields; full-text search can support text-oriented queries when configured. Applications may also maintain their own indexes, such as a key listing all users in a group.

More advanced queries require extra storage, index maintenance, and care when data changes. Therefore, Riak is most efficient when the application knows the key. Confirm available query features against the exact Riak distribution and version being used.

6. Store millions of web-application user sessions in Riak.

Use a bucket or namespace such as “sessions.” Give each session a unique key, for example “session:ab82f.” Store a value such as:

```
{
"user_id": "U204",
"login_time": "2026-10-10T09:30:00Z",
"device": "Android mobile",
"expires_at": "2026-10-10T17:30:00Z"
}
```

The application reads a session by key during a request and updates or deletes it when the user logs out. Configure a suitable expiration/TTL policy if supported by the selected Riak version, or enforce expiry in application logic. Use replication and appropriate read/write quorum settings to tolerate node failures.

Advantages include fast key-based access, flexible session fields, horizontal distribution, and continued service when some nodes are unavailable. Use unique session IDs, secure cookies, encryption where appropriate, and avoid storing passwords or unnecessary sensitive data in session values.

7. Design a Key-Value structure for user profiles, preferences, and shopping carts.

A conceptual design could use separate keys for each logical object:

• “profile:U101” →

```
{"name":"Aman","email":"aman@example.com","city":"Roorkee"}
```

• “preferences:U101” →

```
{"language":"English","theme":"dark","notifications":true}
```

• “cart:U101” →

```
{"items":[{"product_id":"P11","qty":2}
```

,

```
{"product_id":"P28","qty":1}
```

],"updated_at":"2026-10-10T10:00:00Z"}

The application constructs the key from the user ID and performs a direct lookup. Updating preferences means reading and writing the preference value; updating a cart means changing its item list and saving it. Keep related data in one value when it must be fetched together, but split values when they change independently or may grow large.

For concurrent cart updates, use supported conditional/versioned writes or a data type with appropriate update semantics. Validate quantities and prices in application logic, and treat the final checkout as a separate workflow that checks inventory and price again.

8. Explain relationships among data in Key-Value databases.

A simple key-value model primarily maps one key to one value. It does not naturally represent foreign keys, joins, or multi-hop relationships as relational and graph databases do. If a user is connected to many groups, the application may store group IDs in the user value or maintain a separate key such as “user-groups:U101.”

Common techniques include embedding related information in one value, keeping reference IDs between values, maintaining reverse lookup keys or secondary indexes, and using application logic to fetch linked records. Embedding reduces extra lookups but can duplicate data and make updates harder. References reduce duplication but require multiple reads and consistency management.

For heavily connected data with frequent relationship traversal—such as finding friends of friends—a graph database is usually a more natural fit.

9. Explain scaling in Riak.

Riak scales horizontally by distributing data and requests across multiple nodes. Consistent hashing maps keys to partitions in a ring, and partitions are assigned across cluster nodes. When capacity is increased, partitions can be redistributed so new nodes take on part of the workload.

Replication stores copies of data on multiple nodes. If one node fails, another replica may serve a request, improving fault tolerance and availability. Read and write quorum settings control how many replicas must respond. Adding nodes can increase storage capacity and throughput, although actual gains depend on workload, network, replication factor, and cluster configuration.

Scaling also requires monitoring hot keys, balancing partitions, planning capacity, and testing node failure and recovery. Replication improves resilience but uses additional disk space and network bandwidth.

10. Explain Operations by Sets in Key-Value databases.

Set-based operations work with collections of unique elements. Common operations are union (all elements from either set), intersection (elements shared by both), and difference (elements in one set but not the other). Some key-value systems provide native set data types; others store a list/set in a value and process it in application code.

Example: “tags:product:P10” → {"eco-friendly","bestseller","home"}. To find tags common to two products, calculate the intersection. “members:group:G1” can store user IDs, while a set union can combine members of two groups. A social application might maintain a set of followed user IDs.

Set operations simplify membership checks, deduplication, grouping, and tag management. The application must still consider concurrency, atomicity, and index maintenance. Riak’s supported data types vary by version, so the exact set-operation API should be checked for the deployed version.

11. Explain Graph Databases and their data model.

A graph database stores data as nodes and relationships (edges). Nodes represent entities such as people, products, or places. Relationships connect nodes and have a type, such as FRIEND_OF, BOUGHT, or FOLLOWS. Nodes and relationships can both have properties, which are key-value attributes. Labels classify nodes, for example :User or :Product.

In Neo4j, a small example is (Alice:User)-[:FOLLOWS]->(Bob:User). The nodes may have name and age properties; the relationship may have a created_at property. Relationships are stored as first-class elements, allowing the database to follow connections directly.

Graph databases are useful when the value of the data lies in its connections. They support traversals such as friends of friends, recommendation paths, fraud networks, and route finding without requiring repeated relational joins for every relationship hop.

12. Explain major features of Neo4j as a Graph NoSQL database.

Neo4j is a property-graph database with labeled nodes, typed relationships, and properties. It supports the Cypher query language for creating, reading, updating, deleting, and traversing graph patterns. Neo4j provides ACID transactions for supported database operations, helping related changes commit or roll back together.

Neo4j prioritizes transactional consistency rather than the highly available, eventually reconciled model associated with some distributed key-value stores. Availability and scaling depend on the product edition, deployment architecture, and configuration. Clustered deployments can use replication and routing for resilience and read scaling; write scaling and sharding are more constrained and deployment-specific than in systems designed primarily for horizontal key-value distribution.

Compared with a relational database, Neo4j stores relationships directly and can make multi-hop connected-data queries more natural. Relational databases remain strong for structured tabular data, standard reporting, and workloads that fit tables and joins well.

13. Explain graph query processing using Neo4j.

Neo4j queries often describe a pattern of nodes and relationships using Cypher. For example:

```
MATCHMATCH;
```

14. Model a social networking application using Neo4j.

Represent each person as a :User node, each post as a :Post node, and each community as a :Group node. Use relationships such as:

(:User)-[:FRIENDS_WITH]-(:User)
(:User)-[:FOLLOWS]->(:User)
(:User)-[:POSTED]->(:Post)
(:User)-[:LIKED]->(:Post)
(:User)-[:MEMBER_OF]->(:Group)

Store properties such as user_id, name, post text, created_at, and group name. A friendship can be represented as a bidirectional logical connection or as two directed relationships, depending on application rules. Followers are naturally directional.

Neo4j can query a user’s followers, mutual friends, group members, popular posts, and paths connecting users. Recommendations can identify posts liked by similar users or friends-of-friends who follow a topic. Graph modeling keeps relationships explicit and makes multi-hop questions easier to express. Access controls, privacy rules, and recommendation ranking still need application-level design.

15. Compare Riak Key-Value databases and Neo4j Graph databases.

Riak stores values addressed by keys and is well suited to direct lookups, distributed storage, and high availability. Neo4j stores a property graph—nodes, relationships, labels, and properties—and is suited to traversing complex connections.

• Data structure: Riak uses key-value objects; Neo4j uses nodes and typed edges.
• Consistency: Riak offers configurable replica read/write behavior and may require conflict resolution; Neo4j emphasizes transactional consistency.
• Transactions: Riak is centered on key-level operations and application-managed multi-step workflows; Neo4j supports ACID transactions for supported operations.
• Availability: Riak is designed to remain available across node failures; Neo4j availability depends on its cluster/deployment configuration.
• Queries: Riak is strongest when the key is known, with optional indexes/search depending on version; Neo4j uses Cypher for graph-pattern and path queries.
• Relationships: Riak requires references, embedded data, or application-maintained indexes; Neo4j represents relationships directly.
• Scaling: Riak distributes keys and replicas across nodes; Neo4j supports clustered deployments, but horizontal write scaling/sharding is more deployment-specific.
• Use cases: Riak suits sessions, cache-like data, profiles, and key-addressed carts. Neo4j suits social graphs, recommendations, fraud analysis, and network exploration.

Recommendation: a shopping cart can use Riak when the cart is retrieved by customer/session key and needs simple fast updates; checkout correctness must be handled carefully. User profiles and preferences can also fit Riak when accessed by user ID. A social-network recommendation system is better suited to Neo4j because friends, follows, likes, and shared interests form a highly connected graph. The final choice should follow consistency, query, and operational requirements rather than database category alone.